

LENDING ANALYTICS DESIGN

1. Problem Overview

- Objective: Establish a robust Lending Analytics capability for the Personal Loan product to inform funnel performance, risk decisions, and commercial outcomes across channels (mobile, web, branch). Primary stakeholders include Business (Sales, Marketing, Product), the Lending Product squad (BA/Dev/QA), and dependent consumer teams.
- Business requirements: Build Data Lake, Data Warehouse, and governed pipelines to enable:
 - Funnel analytics: measure application-to-funding conversion and identify customer-journey drop-offs by stage and channel.
 - Conversion diagnostics: quantify approval, offer-acceptance, and disbursement rates with cohorting and attribution.
 - Commercial performance: track sales trends (e.g., disbursement amount, funded-loan count) and mix by product, tenor, and channel.
- Compliance requirements: Protect PII and sensitive data via masking, role-based access, and auditing to meet regulatory and internal-governance standards.
- Non-functional requirements:
 - Real-time and near-real-time use cases for operational monitoring and decisioning.
 - Daily Batch analytics for trend analysis, cohorts, and performance reporting.
- Notes:
 - You can make any assumptions in your solution. But you have to write down your assumption explicitly in your solution.
 - You can designed based on your favourist tech stack. But here is TymeX Tech stack for your references

Category	Tech
Platform	Databricks (on AWS) — unified platform cho ETL, analytics, ML
Architecture	Lakehouse (Medallion: Bronze/Silver/Gold) + Delta Lake
Compute Engine	Apache Spark / PySpark
Language	Python, SQL
ETL/Orchestration	AWS Glue — serverless ETL, data cataloging
Data Governance	Unity Catalog — lineage, access control, data contracts
Streaming	Kafka, Segment (CDP), DMS (CDC from RDS)
Data Sources	DynamoDB, RDS (MySQL/Postgres), S3, Event streams
Library	Pandas — data manipulation & analysis

2. Analytics Common Questions

1. Funnel conversion: for applications submitted in a given month, what % reached FUNDED within 14 days? Break it down by channel (mobile / web / branch) and by why the rest didn't — credit decline vs withdrawn / expired / failed-disbursement / offer-not-accepted.
2. Decision reproducibility: for any given `application_number`, reconstruct the exact inputs the credit engine saw at decision time (KYC verdict, bureau score, declared income, employer) — even if the source records have since changed.
3. Offer expiry watchlist: each morning, ops needs offers expiring in the next 24 hours that haven't been accepted yet, plus offers that already expired without action in the past 7 days.
4. A regulator requests proof that PII fields (`national_id`, `mobile_number`...) were masked before any analyst accessed the data for the funnel report in Q1. How would you demonstrate this?"
5. And more....

3. Source Database

GoTyme Bank's Personal Loan product runs on four microservices, each with its own MySQL database. They communicate via REST and async events on Kafka (CDC + domain events), and their databases don't share foreign keys — every cross-service join happens at the application layer (or, now, in the data platform).

3.1. `application-service` — Loan application intake

A team owns this. It captures the applicant profile and loan application — the first step in the funnel.

```
-- application-service database: aurora_los_application
```

```
CREATE TABLE `whitelist` (  
  `id` INT AUTO_INCREMENT PRIMARY KEY,  
  `national_id` VARCHAR(20) NOT NULL,  
  `first_name` VARCHAR(100),  
  `last_name` VARCHAR(100),  
  `mobile_number` VARCHAR(20) NOT NULL,  
  `product_codes` VARCHAR(200) NOT NULL,      -- comma-separated, e.g. 'PL-STD, PL-SAL'  
  `pre_approved_limit` DECIMAL(12,2),  
  `enabled` BIT(1) NOT NULL DEFAULT b'1',  
  `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  `last_updated_date` DATETIME,  
  UNIQUE KEY `uk_whitelist_nid` (`national_id`)  
) ENGINE=InnoDB;  
  
CREATE TABLE `loan_application` (  
  `id` BIGINT AUTO_INCREMENT PRIMARY KEY,  
  `application_number` VARCHAR(30) NOT NULL,  -- e.g. 'AURPL-20260513-00042'  
  `product_code` VARCHAR(20) NOT NULL,  
  `requested_amount` DECIMAL(12,2) NOT NULL,  
  `requested_tenor_months` INT NOT NULL,  
  `loan_purpose` VARCHAR(50),  
  `channel` VARCHAR(20),                      -- 'MOBILE', 'WEB', 'BRANCH';  
  exactly one of applicant / branch_applicant points back to this row
```

```

        `branch_code` VARCHAR(20), -- NULL for non-branch channels
        `status` VARCHAR(30) NOT NULL, -- 'APPLYING', 'SUBMITTED',
'CONDITIONALLY_APPROVED', 'APPROVED', 'DECLINED', 'OFFER_RETRACTED',
'CONTRACT_SIGNED', 'FUNDED', 'APPLYING_EXPIRED', 'WITHDRAWN'
        `submitted_at` DATETIME, -- first submission
        `last_submitted_at` DATETIME, -- most recent re-submission
(after edit)
        `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
        `last_updated_date` DATETIME,
        UNIQUE KEY `uk_application_number` (`application_number`)
) ENGINE=InnoDB;

```

```

CREATE TABLE `applicant` (
        `id` VARCHAR(36) NOT NULL PRIMARY KEY, -- UUID
        `loan_application_id` BIGINT NOT NULL, -- digital channel; 1:1 with
loan_application
        `customer_id` VARCHAR(36) NOT NULL, -- UUID; non-unique
        `national_id` VARCHAR(20) NOT NULL,
        `first_name` VARCHAR(100),
        `last_name` VARCHAR(100),
        `mobile_number` VARCHAR(20),
        `email` VARCHAR(200),
        `date_of_birth` DATE,
        `marital_status` VARCHAR(30), -- free-text; see list_of_value
        `employment_status` VARCHAR(30),
        `monthly_gross_income` DECIMAL(12,2),
        `employer_name` VARCHAR(200),
        `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
        `last_updated_date` DATETIME,
        UNIQUE KEY `uk_applicant_loan_app` (`loan_application_id`),
        KEY `idx_applicant_customer_id` (`customer_id`),
        CONSTRAINT `fk_applicant_loan_app` FOREIGN KEY (`loan_application_id`)
        REFERENCES `loan_application` (`id`)
) ENGINE=InnoDB;

```

```

CREATE TABLE `branch_applicant` (
        `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
        `loan_application_id` BIGINT NOT NULL, -- branch channel; 1:1 with
loan_application
        `national_id` VARCHAR(20) NOT NULL,
        `first_name` VARCHAR(100),
        `last_name` VARCHAR(100),
        `mobile_number` VARCHAR(20),
        `employment_status` VARCHAR(30),
        `monthly_gross_income` DECIMAL(12,2),
        `branch_code` VARCHAR(20) NOT NULL,
        `staff_id` VARCHAR(50), -- branch-staff-assisted
submission
        `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
        `last_updated_date` DATETIME,
        UNIQUE KEY `uk_branch_applicant_loan_app` (`loan_application_id`),
        KEY `idx_branch_applicant_nid` (`national_id`),
        CONSTRAINT `fk_branch_applicant_loan_app` FOREIGN KEY
(`loan_application_id`)
        REFERENCES `loan_application` (`id`)
) ENGINE=InnoDB;

```

```

CREATE TABLE `application_status_history` (
        `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
        `loan_application_id` BIGINT NOT NULL,
        `from_status` VARCHAR(30),
        `to_status` VARCHAR(30) NOT NULL,
        `changed_by` VARCHAR(100), -- 'SYSTEM' or user id

```

```

    `changed_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT `fk_status_history_app` FOREIGN KEY (`loan_application_id`)
        REFERENCES `loan_application` (`id`)
) ENGINE=InnoDB;

CREATE TABLE `list_of_value` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `type` VARCHAR(40) NOT NULL,                -- 'MARITAL_STATUS',
    'EMPLOYMENT_STATUS', 'LOAN_PURPOSE', 'BANK', 'EMPLOYER_SECTOR'
    `code` VARCHAR(50) NOT NULL,
    `display_name` VARCHAR(200),
    `parent_id` BIGINT,                        -- self-FK for hierarchies (e.g.
    EMPLOYER_SECTOR > sub-sector)
    `metadata_json` TEXT,                      -- per-domain blob
    `enabled` BIT(1) NOT NULL DEFAULT b'1',
    `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY `uk_lov_type_code` (`type`, `code`),
    CONSTRAINT `fk_lov_parent` FOREIGN KEY (`parent_id`) REFERENCES
    `list_of_value` (`id`)
) ENGINE=InnoDB;

```

3.2. kyc-service — Identity & bank-statement verification

B team owns this. It runs after application-service submits, produces a KYC verdict, and feeds that into the decisioning service.

```

-- kyc-service database: aurora_los_kyc

CREATE TABLE `kyc_request` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `user_ref` VARCHAR(50) NOT NULL,            -- format: 'CUST-<uuid>'
    `application_reference` VARCHAR(30) NOT NULL, -- foreign application
    number
    `kyc_type` VARCHAR(20) NOT NULL,            -- 'FULL', 'REFRESH', 'RE_SUBMIT'
    `status` VARCHAR(30) NOT NULL,              -- 'PENDING', 'IN_REVIEW',
    'COMPLETED', 'FAILED'
    `requested_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    `completed_at` DATETIME,
    UNIQUE KEY `uk_kyc_app_ref` (`application_reference`, `kyc_type`)
) ENGINE=InnoDB;

CREATE TABLE `document_upload` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `kyc_request_id` BIGINT NOT NULL,
    `document_type` VARCHAR(30) NOT NULL,        -- 'ID_FRONT', 'ID_BACK',
    'PAYSIP', 'BANK_STMT'
    `s3_path` VARCHAR(500) NOT NULL,
    `uploaded_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    `verified_at` DATETIME,                     -- auto-verification timestamp
    `verification_completed_at` DATETIME,        -- ops-agent manual completion
    `verification_status` VARCHAR(20),          -- 'PASS', 'FAIL',
    'MANUAL_REVIEW'
    CONSTRAINT `fk_doc_kyc` FOREIGN KEY (`kyc_request_id`)
        REFERENCES `kyc_request` (`id`)
) ENGINE=InnoDB;

CREATE TABLE `bank_statement_parse` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `kyc_request_id` BIGINT NOT NULL,
    `provider` VARCHAR(30),                     -- 'TRUID', 'MANUAL_PDF'

```

```

    `statement_start_date` DATE,
    `statement_end_date` DATE,
    `declared_monthly_income` DECIMAL(12,2),
    `observed_avg_monthly_deposits` DECIMAL(12,2),
    `consent_given_at` DATETIME,
    CONSTRAINT `fk_bsp_kyc` FOREIGN KEY (`kyc_request_id`)
        REFERENCES `kyc_request` (`id`)
) ENGINE=InnoDB;

CREATE TABLE `kyc_result` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `kyc_request_id` BIGINT NOT NULL,
    `overall_status` VARCHAR(20) NOT NULL,          -- 'PASS', 'FAIL', 'REFER'
    `decision_outcome` VARCHAR(100),                -- free-text summary, sometimes
differs from overall_status
    `risk_flags` TEXT,                              -- pipe-delimited:
'PEP|ADVERSE_MEDIA'
    `completed_at` DATETIME,
    CONSTRAINT `fk_kyc_result_req` FOREIGN KEY (`kyc_request_id`)
        REFERENCES `kyc_request` (`id`)
) ENGINE=InnoDB;

```

3.3. credit-decision-service — Affordability & approval engine

B team owns this. It consumes KYC output and bureau data, produces a credit decision per application, and can re-decision.

-- credit-decision-service database: aurora_los_decisioning

```

CREATE TABLE `product` (
    `id` INT AUTO_INCREMENT PRIMARY KEY,
    `product_key` VARCHAR(30) NOT NULL,             -- 'STD_PERSONAL_LOAN',
'SALARY_ADVANCE', 'CREDIT_BUILDER'
    `display_name` VARCHAR(100) NOT NULL,
    `min_amount_cents` BIGINT NOT NULL,
    `max_amount_cents` BIGINT NOT NULL,
    `min_tenor_months` INT,
    `max_tenor_months` INT,
    UNIQUE KEY `uk_product_key` (`product_key`)
) ENGINE=InnoDB;

CREATE TABLE `bureau_check` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `application_number` VARCHAR(30) NOT NULL,
    `bureau_provider` VARCHAR(30),                 -- CIC
    `bureau_score` INT,
    `bureau_response_json` TEXT,
    `fetched_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE `credit_assessment` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `applicant_id` VARCHAR(36) NOT NULL,
    `application_number` VARCHAR(30) NOT NULL,
    `product_id` INT NOT NULL,
    `decision_version` INT NOT NULL DEFAULT 1,
    `decision` VARCHAR(20) NOT NULL,               -- 'APPROVED', 'DECLINED'
    `approved_amount_cents` BIGINT,                -- nullable when declined
    `approved_tenor_months` INT,
    `approved_interest_rate_bps` INT,              -- basis points: 2500 = 25.00%

```

```

`decision_reason_code` VARCHAR(30),
`affordability_ratio` DECIMAL(5,4),          -- e.g. 0.3200
`mandate_debit_day` INT,
`mandate_account_hash` VARCHAR(128),
`employment_verified_at` DATETIME,
`income_verified_at` DATETIME,
`origination_fee_cents` BIGINT,
`monthly_fee_cents` BIGINT,
`channel` VARCHAR(20),
`employer_name` VARCHAR(200),
`employer_sector` VARCHAR(50),
`decided_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE KEY `uk_decision_app_version` (`application_number`,
`decision_version`),
KEY `idx_ca_applicant` (`applicant_id`),
CONSTRAINT `fk_ca_product` FOREIGN KEY (`product_id`) REFERENCES `product`
(`id`)
) ENGINE=InnoDB;

CREATE TABLE `loan_decision_v1` (
`id` BIGINT AUTO_INCREMENT PRIMARY KEY,
`application_number` VARCHAR(30),
`decision` VARCHAR(20),
`approved_amount` DECIMAL(12,2),
`decided_at` DATETIME,
KEY `idx_legacy_app_number` (`application_number`)
) ENGINE=InnoDB;
-- DEPRECATED (V1.3.0, 2025-08-14). Historical rows retained for regulator
reproducibility until 2028-08.

```

3.4. disbursement-service — Offer, acceptance, payout

C team owns this. It takes approved credit decisions, generates offers, handles acceptance, executes disbursement, and builds the installment schedule.

```

-- disbursement-service database: aurora_loos_disbursement

CREATE TABLE `loan_offer` (
`id` BIGINT AUTO_INCREMENT PRIMARY KEY,
`borrower_id` VARCHAR(36) NOT NULL,          -- UUID;
`application_number` VARCHAR(30) NOT NULL,
`loan_product` VARCHAR(50) NOT NULL,         -- free-text display name, e.g.
'Personal Loan - Standard'
`principal_amount` DECIMAL(15,2) NOT NULL,
`tenor_months` INT NOT NULL,
`interest_rate_pct` DECIMAL(5,2) NOT NULL,   -- e.g. 25.00
`monthly_instalment` DECIMAL(15,2),
`offered_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
`offer_expires_at` DATETIME NOT NULL,        -- config: offered_at + 7d
`offer_expired_at` DATETIME,                 -- actual expiry if customer
never acted
`is_deleted` BIT(1) NOT NULL DEFAULT b'0',
UNIQUE KEY `uk_offer_app` (`application_number`)
) ENGINE=InnoDB;

CREATE TABLE `offer_acceptance` (
`id` BIGINT AUTO_INCREMENT PRIMARY KEY,
`loan_offer_id` BIGINT NOT NULL,
`accepted_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
`accepted_channel` VARCHAR(20),              -- 'MOBILE', 'WEB', 'BRANCH'

```

```

    `signature_ref` VARCHAR(200),
    CONSTRAINT `fk_acceptance_offer` FOREIGN KEY (`loan_offer_id`)
        REFERENCES `loan_offer` (`id`)
) ENGINE=InnoDB;

CREATE TABLE `disbursement` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `loan_offer_id` BIGINT NOT NULL,
    `disbursed_amount` DECIMAL(15,2) NOT NULL,
    `disbursed_to_account` VARCHAR(30),          -- masked account number
    `disbursement_status` VARCHAR(20) NOT NULL,  -- 'PENDING', 'SUCCESS',
    'FAILED', 'REVERSED'
    `disbursed_at` DATETIME,
    `payment_reference` VARCHAR(100),
    CONSTRAINT `fk_disb_offer` FOREIGN KEY (`loan_offer_id`)
        REFERENCES `loan_offer` (`id`)
) ENGINE=InnoDB;

CREATE TABLE `installment_schedule` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `loan_offer_id` BIGINT NOT NULL,
    `schedule_json` TEXT NOT NULL,                -- JSON: [{period_no, due_date,
principal, interest, total_due}, ...]
    `generated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT `fk_sched_offer` FOREIGN KEY (`loan_offer_id`)
        REFERENCES `loan_offer` (`id`)
) ENGINE=InnoDB;

CREATE TABLE `mandate` (
    `id` BIGINT AUTO_INCREMENT PRIMARY KEY,
    `loan_offer_id` BIGINT NOT NULL,
    `bank_name` VARCHAR(100),
    `account_number_hash` VARCHAR(128),
    `debit_day` INT,                             -- 1-28
    `mandate_status` VARCHAR(20) NOT NULL,        -- 'ACTIVE', 'CANCELLED'
    `created_date` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT `fk_mandate_offer` FOREIGN KEY (`loan_offer_id`)
        REFERENCES `loan_offer` (`id`)
) ENGINE=InnoDB;

```

4. Deliverables

- Data pipeline solution design
- Data Modeling for Bronze, Silver and Gold
- A 1–2 page decision log. Cover at minimum:
 - Identity strategy across the four services
 - How you handled cross-service joins with no enforced FKs
 - Data-governance assumptions (a sentence or two is enough)
 - One trade-off you considered and rejected, and why
 - One thing you'd want to confirm with a source-system owner before shipping